

CNN y GRU con Keras: diseño, evaluación y límites

Informe técnico | Inteligencia Artificial, Licenciatura en Sistemas de Información

1. Objetivo y protocolo

Se implementaron dos tareas independientes: clasificación espacial de dígitos mediante una red convolucional (CNN) y pronóstico de un paso en una serie multivariada mediante una unidad recurrente gated (GRU). Se fijaron semillas, se mantuvieron conjuntos de prueba separados y se exportaron los modelos completos y sus pesos. Los conjuntos elegidos permiten ejecutar el trabajo sin servicios externos.

2. Modulo CNN: datos y partición

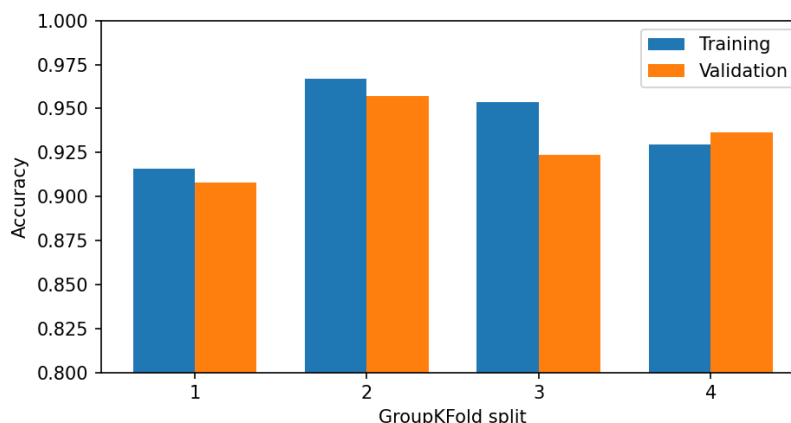
Se utilizaron 1797 imágenes de dígitos manuscritos de scikit-learn, con diez clases. Cada imagen original de 8 x 8 se interpola a 32 x 32 y se asigna una sola vez a uno de doce lotes simulados, equilibrados aproximadamente por clase. Cada lote recibe parámetros propios de brillo, contraste y desenfoque, más ruido reproducible por muestra. Los lotes 0 a 7 forman desarrollo; los lotes 8 a 11 se reservan para prueba. Dentro de desarrollo, GroupKFold de cuatro particiones deja dos lotes distintos para validación en cada iteración. Ninguna imagen ni identificador de lote cruza cada frontera.

Esta simulación somete al clasificador a cambios fotograficos. El conjunto no ofrece identificadores de autor ni lotes reales de captura: la validación por grupos no demuestra generalización a escritores o dispositivos nuevos. La interpolación aumenta la resolución de entrada, pero no crea información visual adicional.

3. Arquitectura CNN y elecciones

La convolución discreta calcula cada activación como suma ponderada sobre una vecindad local de 3 x 3. Con stride $S=1$ y padding 'same', la altura de salida es $\text{ceil}(H/S)$, por lo que se conserva la posición de las características dentro de cada bloque. Los filtros pasan de 32 a 64 y 64 canales para combinar rasgos progresivamente. Dos max-pooling de 2 x 2 y stride 2 reducen mapas de 32 a 16 y luego a 8, aportando tolerancia local a desplazamientos pequeños y reduciendo el costo. La convolución es aproximadamente equivariante a traslaciones en el interior; el pooling introduce cierta invariancia local, no equivarianza exacta.

ReLU evita saturación para entradas positivas; global average pooling reduce parámetros frente a aplanar mapas, y una capa densa de 64 unidades agrega capacidad. Dropout 0,25 mitiga coadaptación. La salida softmax de diez clases se ajusta con entropía cruzada categórica dispersa. Adam (tasa 0,001; batch 32) usa clipnorm 1,0 para limitar gradientes extremos. En cada fold, early stopping con paciencia 5 restituye los mejores pesos según pérdida de validación.



Exactitud de entrenamiento y validación por fold con lotes excluyentes.

4. Modulo GRU: serie y corte temporal

Se generó una serie sintetica de 1600 instantes con valor autorregresivo, seno y coseno estacionales, un control correlacionado y ruido. La variable objetivo en t depende de su valor anterior, de la estación en t y del control en t-1. Cada entrada contiene 36 pasos anteriores y cuatro variables; el objetivo es el valor del paso siguiente. Los tramos son entrenamiento [0,960), validación [960,1280) y prueba [1280,1600). Todas las ventanas se construyen totalmente dentro de su propio tramo; se descartan los primeros 36 objetivos en cada frontera para que ninguna ventana de validación o prueba comparta historia con el tramo previo.

La media y desviación de las cuatro entradas se ajustan exclusivamente con las ventanas de entrenamiento; el objetivo se escala con sus valores de entrenamiento. Los parámetros se guardan en gru_scalers.npz. El modelo de referencia por persistencia predice el último valor observado. La serie es un ejemplo controlado: estas métricas no prueban rendimiento sobre pronósticos reales.

5. Arquitectura GRU y estabilidad del gradiente

Una GRU de 32 unidades resume la ventana y alimenta una capa densa ReLU de 16 unidades y una salida lineal, apropiada para regresión sin límite artificial de rango. Las compuertas sigmoid de actualización y reinicio modulan el estado. En forma simplificada, $h_t = z_t h_{(t-1)} + (1-z_t) h_{\text{candidato}}$: el término z_t conserva memoria previa y ofrece una vía aditiva para el gradiente durante BPTT, que puede atenuar su desaparición frente a una RNN simple. No elimina matemáticamente toda desaparición. Se usa tanh acotada para el estado, MSE como pérdida, Adam con tasa 0,001, clipnorm 1,0 para contener explosión del gradiente y early stopping con paciencia 7. El orden de las ventanas se conserva (shuffle=False).

6. Resultados observados: CNN

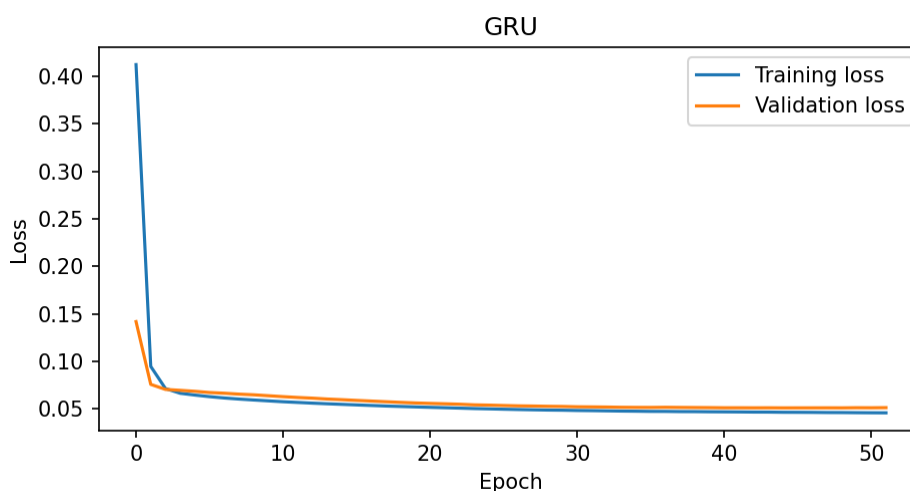
Medida	Resultado
Desarrollo / prueba	1207 / 590 imágenes
Exactitud validación (media ± desv.)	0.9313 ± 0.0180
Exactitud entrenamiento final / prueba	0.9677 / 0.9593
F1 macro prueba	0.9595
Epocas del ajuste final	24

La exactitud media de entrenamiento en los cuatro folds es 0.9414; la brecha media con validación es 0.0101. La desviación entre folds de validación (0,0180) indica sensibilidad moderada a los lotes. Tres brechas son positivas y pequeñas; la tercera alcanza 0,0300. No se observa una brecha sistemática grande de sobreajuste ni exactitudes bajas en ambos lados que indiquen sesgo alto. El modelo final usa más datos; su exactitud de entrenamiento no es directamente comparable con cada fold. La matriz de confusión y los folds completos están en cnn_metrics.json.

7. Resultados observados: GRU

Medida	GRU	Persistencia
MAE prueba	0.0960	0.1139
RMSE prueba	0.1201	0.1428
MAE entrenamiento / validación	0.0841 / 0.0891	-
Mejor época	45	-

La GRU reduce el MAE de persistencia en 15.7%. La brecha entre MAE de entrenamiento y validación es 0.0050: pequeña frente al error medio. MAE por cuarto cronológico de prueba: 0.0987, 0.1002, 0.0947, 0.0902. No se observa crecimiento sostenido del error. La curva permite inspeccionar la separación entre entrenamiento y validación para detectar varianza alta, y pérdidas altas en ambos para detectar sesgo.



Perdida MSE por época en entrenamiento y validación cronológica.

8. Limitaciones e interpretación

La CNN se evalúa en un holdout de lotes simulados, no en nuevas personas. La GRU ve un proceso sintético estacionario definido por reglas conocidas y no puede extrapolarse directamente a un dominio aplicado. Se seleccionan épocas y se detiene el entrenamiento usando solo desarrollo o validación; cada prueba se usa una vez después de fijar pesos. No se reclaman conclusiones causales ni incertidumbre estadística para otras poblaciones. Los resultados son reproducibles con el código, pero detalles de CPU y bibliotecas pueden introducir pequeñas diferencias numéricas.

9. Archivos y reproducción

Ejecutar `pip install -r requirements.txt`, `python train.py`, `python predict.py` y `python -m unittest discover -s tests -v`. Los modelos `cnn.keras` y `gru.keras` contienen arquitectura y pesos; los archivos `*.weights.h5` son pesos adicionales. `gru_scalers.npz` se necesita para interpretar predicciones de la serie. `cnn_metrics.json` y `gru_metrics.json` conservan los resultados, y `gru_learning_curve.png` representa la pérdida por época. El informe se regenera con `python make_report.py`.

Referencias de implementación

Keras API: Conv2D, MaxPooling2D, GRU, EarlyStopping, model saving (keras.io/api/). Scikit-learn: load_digits y GroupKFold (scikit-learn.org/stable/). Formulaciones de arquitecturas y métodos: Goodfellow, Bengio y Courville, Deep Learning, MIT Press, 2016, capítulos 9 y 10.